T3000 API & Protocol

Device Management API · built Nov 13, 2026

Device Management API

Overview

The Device Management API allows you to discover, connect to, and manage BACnet devices on your network. Use these endpoints to scan for devices, check their online status, and retrieve device information.

Common Operations

List All Devices

Get all registered devices with their connection status:

TYPESCRIPT

```typescript
const devices = await fetch('http://localhost:9103/api/t3_device/devices')
  .then(res => res.json());

console.log(devices);
// [
//   {
//     "id": 1,
//     "serial_number": 123456,
//     "product_name": "T3-BB",
//     "ip_address": "192.168.1.100",
//     "online": true
//   }
// ]
```

Get Single Device

Retrieve detailed information about a specific device:

TYPESCRIPT

```typescript
const device = await fetch('http://localhost:9103/api/t3_device/devices/1')
  .then(res => res.json());
```

Add New Device

Register a device manually:

TYPESCRIPT

```typescript
const newDevice = await fetch('http://localhost:9103/api/t3_device/devices', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    serial_number: 654321,
    product_name: 'T3-LB',
    ip_address: '192.168.1.101'
  })
}).then(res => res.json());
```

Update Device

Modify device information:

TYPESCRIPT

```typescript
await fetch('http://localhost:9103/api/t3_device/devices/1', {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    ip_address: '192.168.1.200',
    notes: 'Relocated to second floor'
  })
});
```

Delete Device

Remove a device from the system:

TYPESCRIPT

```typescript
await fetch('http://localhost:9103/api/t3_device/devices/1', {
  method: 'DELETE'
});
```

Check Device Capabilities

Get device features and supported objects:

TYPESCRIPT

```typescript
const capabilities = await fetch('http://localhost:9103/api/t3_device/devices/1/capabili
  .then(res => res.json());

console.log(capabilities);
// {
//   "inputs": 16,
//   "outputs": 8,
//   "variables": 64,
//   "programs": 8,
//   "supports_trendlogs": true
// }
```

React Component Usage

Device List Component

TYPESCRIPT

```typescript
import { useDeviceStore } from '@/stores/deviceStore';

export function DeviceList() {
  const devices = useDeviceStore(state => state.devices);
  const loadDevices = useDeviceStore(state => state.loadDevices);

  useEffect(() => {
    loadDevices();
  }, [loadDevices]);

  return (
    <div>
      {devices.map(device => (
        <DeviceCard key={device.id} device={device} />
      ))}
    </div>
  );
}
```

Zustand Store

TYPESCRIPT

```typescript
// stores/deviceStore.ts
import create from 'zustand';

interface DeviceStore {
  devices: Device[];
  loadDevices: () => Promise<void>;
  addDevice: (device: Partial<Device>) => Promise<void>;
}

export const useDeviceStore = create<DeviceStore>((set) => ({
  devices: [],

  loadDevices: async () => {
    const response = await fetch('http://localhost:9103/api/t3_device/devices');
    const devices = await response.json();
    set({ devices });
  },

  addDevice: async (device) => {
    const response = await fetch('http://localhost:9103/api/t3_device/devices', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(device)
    });
    const newDevice = await response.json();
    set(state => ({ devices: [...state.devices, newDevice] }));
  }
}));
```

Device Management Endpoints

GET /api/t3_device/devices

Description: Retrieve all devices with statistics

Request:

HTTP

```
GET /api/t3_device/devices HTTP/1.1
Host: localhost:9103
Accept: application/json
```

Response:

JSON


```
[
  {
    "id": 1,
    "serial_number": 123456,
    "product_name": "T3-BB",
    "product_id": 211,
    "ip_address": "192.168.1.100",
    "port": 47808,
    "online": true,
    "modbus_id": 1,
    "baudrate": 19200,
    "protocol": "bacnet_ip",
    "panel_number": 5,
    "firmware_version": "95.2",
    "hardware_version": "2.0",
    "location": "Building 1 - 2nd Floor",
    "notes": "",
    "created_at": "2026-01-01T10:00:00Z",
    "updated_at": "2026-01-09T08:30:00Z",
    "last_seen": "2026-01-09T08:29:45Z",
    "stats": {
      "inputs": 16,
      "outputs": 8,
      "variables": 64,
      "programs": 4,
      "schedules": 12
    }
  }
]
```


Used By:

- DeviceTree.tsx
- DeviceList.tsx

Handler: `api/src/t3_device/routes.rs::get_devices_with_stats`

Errors:

- `500` - Database error

GET /api/t3_device/devices/:id

Description: Get single device by ID

Parameters:

- `id` (path, integer) - Device ID

Request:

```HTTP
GET /api/t3_device/devices/1 HTTP/1.1
```

Response:

```JSON
{
  "id": 1,
  "serial_number": 123456,
  "product_name": "T3-BB",
  "ip_address": "192.168.1.100",
  "online": true
  // ... all device fields
}
```

Used By:

- DeviceDetail.tsx

Errors:

- `404` - Device not found

POST /api/t3_device/devices

Description: Create new device

Request:

```HTTP
POST /api/t3_device/devices HTTP/1.1
Content-Type: application/json

{
  "serial_number": 654321,
  "product_name": "T3-LB",
  "product_id": 213,
  "ip_address": "192.168.1.101",
  "port": 47808,
  "modbus_id": 2,
  "baudrate": 19200,
  "location": "Building 2"
}
```

Required Fields:

- `serial_number` (integer)
- `product_name` (string)

Optional Fields:

- `product_id`, `ip_address`, `port`, `modbus_id`, `baudrate`, `location`, `notes`

Response:

```JSON
{
  "id": 2,
  "serial_number": 654321,
  "product_name": "T3-LB",
  "created_at": "2026-01-09T09:00:00Z"
  // ... all fields
}
```

Used By:

- AddDeviceDialog.tsx

Errors:

- `400` - Invalid request body
- `409` - Device with serial number already exists

PUT /api/t3_device/devices/:id

Description: Update existing device

Parameters:

- `id` (path, integer) - Device ID

Request:

```HTTP
PUT /api/t3_device/devices/1 HTTP/1.1
Content-Type: application/json

{
  "ip_address": "192.168.1.200",
  "location": "Building 1 - 3rd Floor",
  "notes": "Relocated during renovation"
}
```

Response:

```JSON
{
  "id": 1,
  "ip_address": "192.168.1.200",
  "location": "Building 1 - 3rd Floor",
  "updated_at": "2026-01-09T09:15:00Z"
  // ... all fields
}
```

Used By:

- EditDeviceDialog.tsx

Errors:

- `404` - Device not found
- `400` - Invalid request body

DELETE /api/t3_device/devices/:id

Description: Delete device and all associated data

Parameters:

- `id` (path, integer) - Device ID

Request:

```HTTP
DELETE /api/t3_device/devices/1 HTTP/1.1
```

Response:

```HTTP
HTTP/1.1 204 No Content
```

Warning: This cascades and deletes all inputs, outputs, variables, programs, schedules, and trendlogs associated with the device.

Used By:

- DeleteDeviceDialog.tsx

Errors:

- `404` - Device not found

GET /api/t3_device/devices/:id/capabilities

Description: Get device capabilities and supported features

Parameters:

- `id` (path, integer) - Device ID

Request:

```HTTP
GET /api/t3_device/devices/1/capabilities HTTP/1.1
```

Response:

JSON


```
{
  "device_id": 1,
  "inputs": {
    "count": 16,
    "max": 32,
    "types": ["analog", "binary"]
  },
  "outputs": {
    "count": 8,
    "max": 16,
    "types": ["analog", "binary"]
  },
  "variables": {
    "count": 64,
    "max": 200
  },
  "programs": {
    "count": 4,
    "max": 8
  },
  "schedules": {
    "count": 12,
    "max": 50
  },
  "trendlogs": {
    "supported": true,
    "count": 6,
    "max": 8
  },
  "features": {
    "bacnet_mstp": false,
    "bacnet_ip": true,
    "modbus": true,
    "web_interface": true
  }
}
```


Used By:

- DeviceCapabilities.tsx

Errors:

- `404` - Device not found

GET /api/t3_device/devices/count

Description: Get total device count

Request:

```HTTP
GET /api/t3_device/devices/count HTTP/1.1
```

Response:

```JSON
{
  "count": 12
}
```

Used By:

- Dashboard.tsx

POST /api/t3_device/devices/:id/refresh

Description: Force refresh device from hardware (via BACnet)

Parameters:

- `id` (path, integer) - Device ID

Request:

```HTTP
POST /api/t3_device/devices/1/refresh HTTP/1.1
```

Response:

JSON

```
{
  "success": true,
  "device_id": 1,
  "refreshed_at": "2026-01-09T09:30:00Z",
  "objects_updated": 142
}
```

Used By:

- RefreshDeviceButton.tsx

Errors:

- `404` - Device not found
- `503` - Device offline or unreachable

GET /api/t3_device/devices/:id/connection-status

Description: Check real-time device connection status

Parameters:

- `id` (path, integer) - Device ID

Request:

HTTP

```
GET /api/t3_device/devices/1/connection-status HTTP/1.1
```

Response:

JSON

```
{
  "online": true,
  "last_seen": "2026-01-09T09:29:45Z",
  "response_time_ms": 45,
  "connection_type": "bacnet_ip",
  "errors": []
}
```

Used By:

- DeviceStatusIndicator.tsx

Errors:

- `404` - Device not found

Implementation Details

Database Schema

```SQL
CREATE TABLE DEVICES (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    serial_number INTEGER NOT NULL UNIQUE,
    product_name TEXT NOT NULL,
    product_id INTEGER,
    ip_address TEXT,
    port INTEGER DEFAULT 47808,
    online INTEGER DEFAULT 0,
    modbus_id INTEGER,
    baudrate INTEGER DEFAULT 19200,
    protocol TEXT DEFAULT 'bacnet_ip',
    panel_number INTEGER,
    firmware_version TEXT,
    hardware_version TEXT,
    location TEXT,
    notes TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    last_seen DATETIME
);

CREATE INDEX idx_devices_serial ON DEVICES(serial_number);
CREATE INDEX idx_devices_online ON DEVICES(online);
```

Rust Handler Example

RUST

```rust
// api/src/t3_device/routes.rs
async fn get_devices_with_stats(
    State(state): State<T3AppState>,
) -> Result<Json<Vec<DeviceWithStats>>, ApiError> {
    let conn = state.get_conn().await?;

    let devices = Device::find()
        .all(&conn)
        .await
        .map_err(|e| ApiError::DatabaseError(e.to_string()))?;

    let mut result = Vec::new();

    for device in devices {
        let stats = get_device_stats(&conn, device.id).await?;
        result.push(DeviceWithStats {
            device,
            stats,
        });
    }

    Ok(Json(result))
}

async fn get_device_stats(conn: &DatabaseConnection, device_id: i32) -> Result<DeviceSta
    let inputs = Input::find()
        .filter(input::Column::DeviceId.eq(device_id))
        .count(conn)
        .await?;

    let outputs = Output::find()
        .filter(output::Column::DeviceId.eq(device_id))
        .count(conn)
        .await?;

    // ... count other objects

    Ok(DeviceStats {
        inputs: inputs as i32,
        outputs: outputs as i32,
```

```
        // ...
    })
}
```

TypeScript Types

TYPESCRIPT

```typescript
// types/device.ts
export interface Device {
  id: number;
  serial_number: number;
  product_name: string;
  product_id?: number;
  ip_address?: string;
  port?: number;
  online: boolean;
  modbus_id?: number;
  baudrate?: number;
  protocol?: string;
  panel_number?: number;
  firmware_version?: string;
  hardware_version?: string;
  location?: string;
  notes?: string;
  created_at: string;
  updated_at: string;
  last_seen?: string;
}

export interface DeviceStats {
  inputs: number;
  outputs: number;
  variables: number;
  programs: number;
  schedules: number;
}

export interface DeviceWithStats {
  device: Device;
  stats: DeviceStats;
}
```

Error Codes

- `DEVICE_NOT_FOUND` - Device ID doesn't exist
- `DEVICE_OFFLINE` - Device not reachable

- `DUPLICATE_SERIAL` - Serial number already registered
- `DATABASE_ERROR` - Internal database error
- `INVALID_REQUEST` - Malformed request body